

The background of the slide features a large, light gray watermark of the University of Otago logo. The logo consists of a stylized, circular emblem with multiple curved lines radiating from the center, creating a sense of motion or a globe. Above the emblem is a horizontal bar.

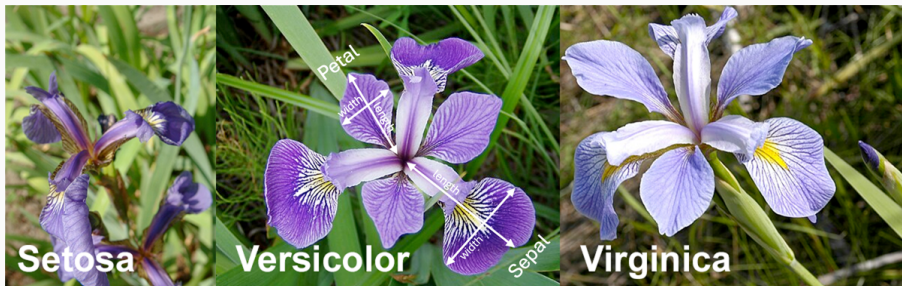
COSC343/AIML402

MODULE: MACHINE LEARNING

LECTURE 12: CLASSIFICATION II - PERCEPTRON

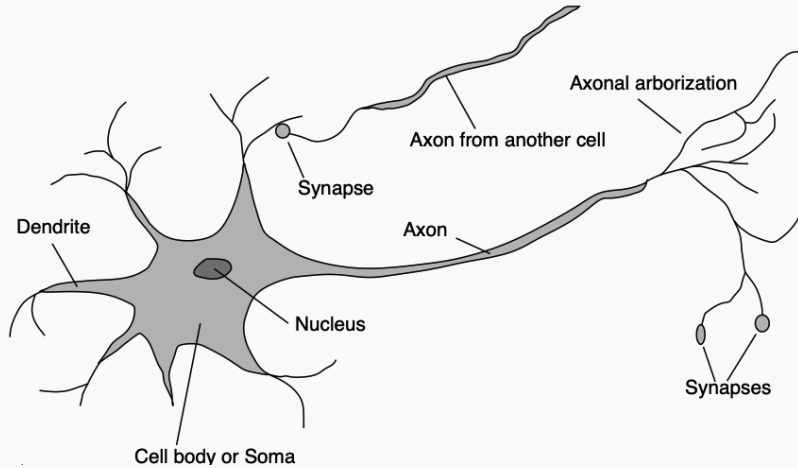
Lech Szymanski

Iris dataset

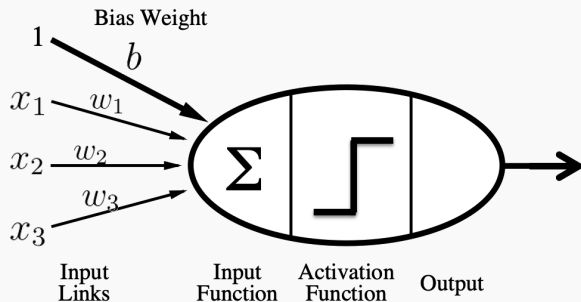


https://en.wikipedia.org/wiki/Iris_flower_data_set

Biological neuron



McCulloch-Pitts unit/Artificial neuron with a hard-limiting function



$$v = x_1w_1 + x_2w_2 + x_3w_3 + b$$

$$\sigma_{\text{hardlim}}(v) = \begin{cases} 1 & v > 0 \\ 0 & v \leq 0 \end{cases}$$

Artificial neuron (vector math)

The hypothesis given by an artificial neuron taking a d -attribute input $x \in \mathcal{R}^d$ and passing it through some **activation** function $\sigma()$ is:

$$\begin{aligned}h(x, w_1, w_2, \dots, b) &= \sigma(x_1 w_1 + x_2 w_2 + \dots + x_d w_d + b) \\&= \sigma\left(\sum_{i=1}^d x_i w_i + b\right) \\&= \sigma(\mathbf{x} \cdot \mathbf{w} + b),\end{aligned}$$

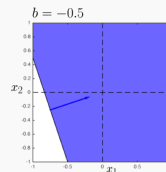
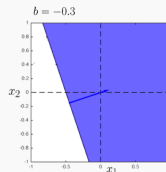
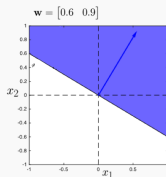
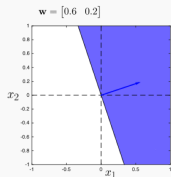
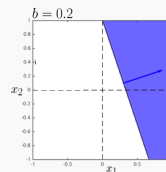
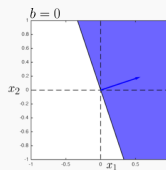
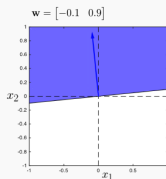
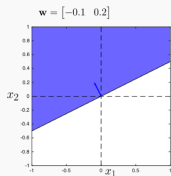
where $\mathbf{x} = \begin{bmatrix} x_1 & x_2 & \dots & x_d \end{bmatrix}$ and $\mathbf{w} = \begin{bmatrix} w_1 & w_2 & \dots & w_d \end{bmatrix}$ are d -dimensional vectors, and $\cdot$ signifies the dot product of vectors.

Artificial neuron (geometric interpretation)

Geometrically-speaking, the single-neuron perceptron divides an input space into two regions, **half-spaces**, where one side are the inputs that cause the neuron to output the value of 1, and the other contains the inputs that cause the neuron to output the value of 0.

$$b = 0$$

$$w = \begin{bmatrix} 0.6 & 0.2 \end{bmatrix}$$



Formalising the perceptron learning rule

Given a set of training data pairs (x, y) , where $x \in \mathcal{R}^d$ and $y \in \{0, 1\}$, and assuming a small value learning parameter $0 < \alpha < 1$:

1. Create a perceptron with a weight vector $\mathbf{w}$ and b initialised to random values.
2. Do
3. Compute the perceptron output for given input $\mathbf{x}$:

$$\hat{y} = \begin{cases} 1 & \mathbf{x} \cdot \mathbf{w} + b > 0 \\ 0 & \mathbf{x} \cdot \mathbf{w} + b \leq 0 \end{cases}$$

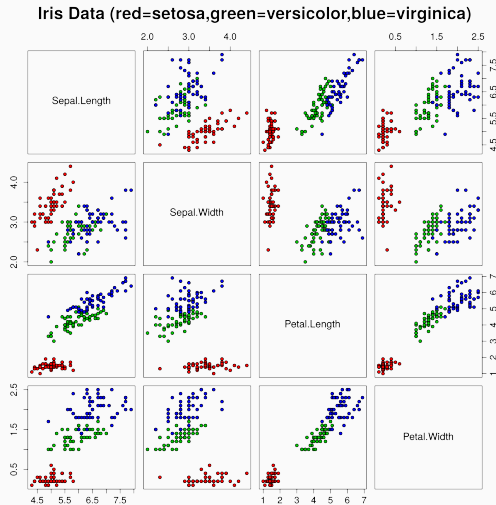
4. Evaluate the error in the output: $e = y - \hat{y}$
5. Update the weights and the bias as follows:

$$w_i := w_i + \alpha e x_i, \forall i = 1, \dots, d$$

$$b := b + \alpha e.$$

While e is not zero for all (x, y)

Example: Iris binary classification and 2D input



Training regimes

The α parameter controls the **learning rate**. The larger the learning rate, the faster the learning, but potentially unstable.

A set of updates as a result of exposure of the learner to the entire dataset is referred to as the training **epoch**.

Given N training pairs $(\mathbf{x}_1, y_1), \dots, (\mathbf{x}_N, y_N)$:

Online learning

One update per training sample, N
updates per epoch:

$$w_i := w_i + \alpha e_n x_{n,i}$$

Batch learning Average updates from all
training samples, One update per epoch:

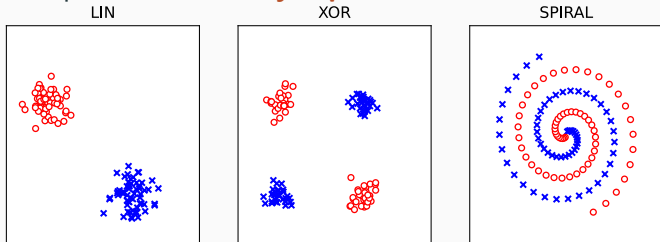
$$w_i := w_i + \frac{1}{N} \sum_{n=1}^N \alpha e_n x_{n,i}$$

Mini-batch learning is the process of dividing the data into fixed-size batches, averaging updates over mini-batch and shuffling the data assignment to mini-batches between the epochs.

Linear separation

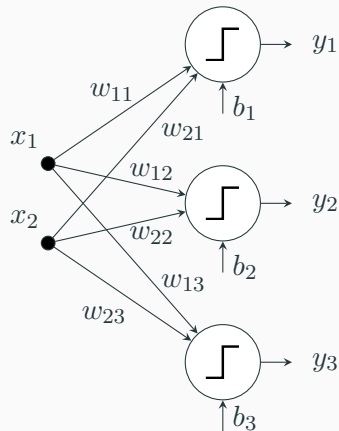
Perceptron learning rule is guaranteed to converge (meaning it will stop updating parameters, because errors for all data point are zero) when the following conditions are satisfied:

- the **learning parameter** is small enough,
the proof for this doesn't specify what small enough means – just rest assured that, whatever the value you choose, there is always a possibility of making it smaller
- the classification problem is **linearly separable**.



Multi-class classification

For **multi-class** problems with K classes (where $K > 2$) we use K neurons and train each to separate one class from all others. We encode the class labels as **one-hot** K -dimensional vectors, where all the entries but one are zero, with the index of non-zero entry corresponding to the class label.



$$c_1 = \begin{bmatrix} 1 & 0 & 0 \end{bmatrix} \quad c_2 = \begin{bmatrix} 0 & 1 & 0 \end{bmatrix} \quad c_3 = \begin{bmatrix} 0 & 0 & 1 \end{bmatrix}$$

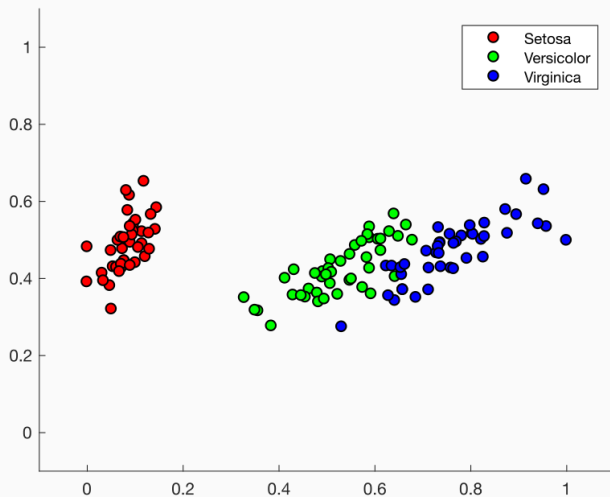
$$y_j = \sigma \left(\sum_{i=1}^d x_i w_{ij} + b_j \right)$$

$$\mathbf{y} = \sigma(\mathbf{x}\mathbf{W} + \mathbf{b}),$$

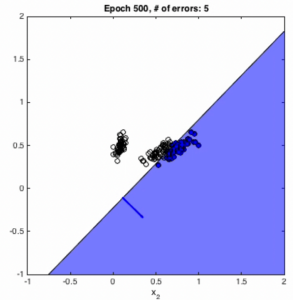
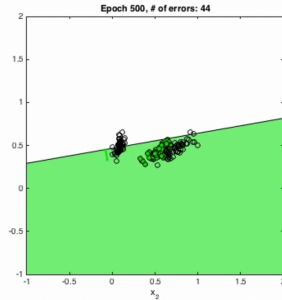
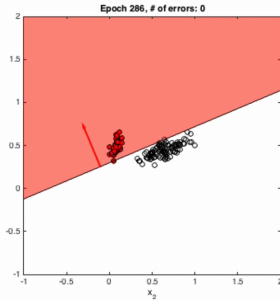
$$\text{where } \mathbf{y} = \begin{bmatrix} y_1 & \dots & y_K \end{bmatrix}, \mathbf{x} = \begin{bmatrix} x_1 & \dots & x_d \end{bmatrix},$$

$$\mathbf{W} = \begin{bmatrix} w_{11} & \dots & w_{1K} \\ \vdots & \ddots & \vdots \\ w_{dK} & \dots & w_{dK} \end{bmatrix}, \mathbf{b} = \begin{bmatrix} b_1 & \dots & b_K \end{bmatrix}$$

Example: Iris 2D



Example: Iris 2D



Lab 6: Iris classification

Objectives:

- Revise and solidify your understanding of the Perceptron learning rule
- Implement the Perceptron learning rule using Numpy library
- Reinforce your understanding of the geometric representation of the Perceptron through visualisations of its decision function
- Train the Perceptron on the Iris dataset

Study guide

- Understand well the architecture of an artificial neuron.
- Terms and definitions: activation function, weights, bias, half-space, one-hot encoding, binary vs. multi-class classification.
- Understand geometric interpretation of the role of the weights and bias played in the perceptron's decision function.
- Be familiar (don't memorise, but understand the math) with the process of training the perceptron; understand the role of the learning parameter.
- Understand the limits of the perceptron model, especially when it comes to linear separation.
- Know how to set-up the perceptron architecture to do multi-class classification.

Reading for this lecture

AIMA Chapter 19, Section 6.4

What's next?

Regression

Read AIMA Chapter 19, Section 6